

NAME: Tomas Gebrewold
ID: 011822853

This project implements three CPU scheduling algorithms: First-Come First-Served (FCFS), Round Robin (RR), and Shortest Job First (SJF). Each process is described using four parameters: "A" (arrival time), "B" (upper bound of CPU burst), "C" (total CPU time needed), and "M" (I/O multiplier). Input is provided in the format "n (A B C M) (A B C M) ..." where "n" indicates the number of processes. Each process is stored as a struct containing various fields, including its state (tracked using boolean flags like "isReady", "isBlocked", "isRunning", and "isFinished") and counters for CPU time, I/O time, and waiting time.

The FCFS implementation begins by sorting the process list based on arrival time "A". A queue is used to manage ready processes. At each cycle, the scheduler checks for newly arrived or I/O-completed processes and places them into the ready queue. If the CPU is idle, the next ready process is dequeued and assigned a CPU burst using the "randomOS(B)" function. A process continues running until the burst ends. If its total required CPU time "C" is reached, it finishes; otherwise, it enters I/O, which lasts for " $M \times \text{last_cpu_burst}$ " cycles. Once I/O is completed, it re-enters the ready queue. Tie-breaking in FCFS is straightforward: if two processes arrive at the same time, the one with the lower index in the input is scheduled first.

The Round Robin scheduler builds upon the FCFS structure but introduces a fixed time quantum—in this case, two cycles. When a process is scheduled, it can run for a maximum of two cycles, even if its burst is longer. If it doesn't finish within the quantum, it is preempted and re-enqueued to allow fair sharing. Processes that complete their burst either finish (if total CPU time "C" is met) or enter I/O. Tie-breaking rules are consistent with FCFS, maintaining input order when processes arrive or complete I/O simultaneously.

The Shortest Job First algorithm does not use a queue. Instead, at each cycle, it scans the list of ready processes and selects the one with the least remaining CPU time, computed as " $C - \text{currentCPUTimeRun}$ ". If there is a tie in remaining time, the process with the lower input index is selected. Once selected, a CPU burst is generated and the process runs to completion of the burst or its total CPU time. If the process is not finished, it enters I/O and becomes ready again afterward. The selection is re-evaluated each time the CPU becomes free.

Across all algorithms, "randomOS" pulls values from a file and uses the formula " $1 + (\text{random_number} \% B)$ " to generate CPU bursts. The offset used in this function ensures each process draws a new number every time it's scheduled. The simulation continues until all processes are marked as finished. Global counters such as "TOTAL_FINISHED_PROCESSES" and "TOTAL_NUMBER_OF_CYCLES_SPENT_BLOCKED" are updated throughout for use in reporting statistics like total turnaround time, CPU utilization, and I/O time.